

Source Code: final.py

5C Network Operations Automation & AI Reporting Engine

```
import os
import csv
import json
import requests
from datetime import datetime
import psycopg2
import clickhouse_connect
from jinja2 import Template
from dotenv import dotenv_values

config = dotenv_values("config.env")
SQL = "sql"

# — DB Helpers —————

def pg_connect(host, port, dbname, user, password):
    try:
        conn = psycopg2.connect(
            host=host, port=int(port or 5432), dbname=dbname,
            user=user, password=password, connect_timeout=5
        )
        print(f" ✓ PostgreSQL @ {host}/{dbname}")
        return conn
    except Exception as e:
        print(f" ✗ PostgreSQL ({host}/{dbname}): {e}")
        return None

def ch_connect():
    try:
        client = clickhouse_connect.get_client(
            host=config.get("CH_HOST"),
            port=int(config.get("CH_PORT", 8123)),
            username=config.get("CH_USER", "default"),
            password=config.get("CH_PASSWORD", ""),
            database=config.get("CH_DB", "default"),
            connect_timeout=5
        )
        print(f" ✓ ClickHouse @ {config.get('CH_HOST')}/{config.get('CH_DB')}")
        return client
    except Exception as e:
        print(f" ✗ ClickHouse: {e}")
        return None

def aura_connect():
    """Connect to the AURA PostgreSQL database."""
    return pg_connect(
        host=config.get("AURA_HOST"),
        port=int(config.get("AURA_PORT", 5432)), # ← cast to int explicitly
        dbname=config.get("AURA_DB"),
        user=config.get("AURA_USER"),
        password=config.get("AURA_PASSWORD")
    )

def ch_query(client, filename):
    path = os.path.join(SQL, filename)
```

```

if not client:
    return []
try:
    with open(path) as f:
        query = f.read().strip()
        return client.query(query).result_rows
except Exception as e:
    print(f" [ERROR] {path}: {e}")
    return []

def pg_query(conn, filename):
    path = os.path.join(SQL, filename)
    if not conn:
        return []
    try:
        with open(path) as f:
            query = f.read().strip()
        with conn.cursor() as cur:
            cur.execute(query)
            cols = [d[0] for d in cur.description]
            return [dict(zip(cols, row)) for row in cur.fetchall()]
    except Exception as e:
        print(f" [ERROR] {path}: {e}")
        return []

def pg_query_raw(conn, sql_string):
    """Run an inline SQL string (not from file) and return list of dicts."""
    if not conn:
        return []
    try:
        with conn.cursor() as cur:
            cur.execute(sql_string)
            cols = [d[0] for d in cur.description]
            return [dict(zip(cols, row)) for row in cur.fetchall()]
    except Exception as e:
        print(f" [ERROR] inline query: {e}")
        return []

# — Snapshot helpers —————
SNAPSHOT_FILE = "yesterday_snapshot.json"

def load_snapshot():
    if os.path.exists(SNAPSHOT_FILE):
        try:
            with open(SNAPSHOT_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return {}

def save_snapshot(data: dict):
    snap = {
        "total":          data["case_counts"]["total"],
        "xray":           data["case_counts"]["xray"],
        "ct":             data["case_counts"]["ct"],
        "mri":            data["case_counts"]["mri"],
        "nm":             data["case_counts"]["nm"],
        "supply":         data["supply"]["performance"],
        "iqc":            data["delivery"]["iqc_percentage"],
        "oqc":            data["delivery"]["oqc_percentage"],
        "tickets":        data["ticket_closure"]["total"],
        "tech":           data["tech_count"],
        "sales":          data["onboarded_count"],
        "reactivations":  data["reactivation_count"],
    }

```

```

        "pending_install_names": [i["name"] for i in data["pending_installations"]],
        "installations_done":    data["installations_done"],
        "last_updated_date":     datetime.now().strftime("%Y-%m-%d"),
    }
    try:
        with open(SNAPSHOT_FILE, "w") as f:
            json.dump(snap, f, indent=2)
    except Exception as e:
        print(f" [WARN] Could not save snapshot: {e}")

def compute_deltas(data: dict, yest: dict) -> dict:
    def d(val, key, fallback=None):
        return val - yest.get(key, fallback if fallback is not None else val)
    return {
        "total_vs_yesterday": d(data["case_counts"]["total"], "total"),
        "xray_delta":         d(data["case_counts"]["xray"], "xray"),
        "ct_delta":           d(data["case_counts"]["ct"], "ct"),
        "mri_delta":          d(data["case_counts"]["mri"], "mri"),
        "nm_delta":           d(data["case_counts"]["nm"], "nm"),
        "supply_delta":       round(d(data["supply"]["performance"], "supply"), 1),
        "iqc_delta":          round(d(data["delivery"]["iqc_percentage"], "iqc"), 1),
        "oqc_delta":          round(d(data["delivery"]["oqc_percentage"], "oqc"), 1),
        "tickets_delta":      d(data["ticket_closure"]["total"], "tickets"),
        "tech_delta":         d(data["tech_count"], "tech"),
        "sales_delta":        d(data["onboarded_count"], "sales"),
        "reactivation_delta": d(data["reactivation_count"], "reactivations"),
    }

# — Installation carry-over logic —————
def enrich_installations(pending_installations: list, yesterday: dict) -> tuple:
    today_str = datetime.now().strftime("%Y-%m-%d")
    snapshot_date = yesterday.get("last_updated_date", "")

    yesterday_pending_names = set(yesterday.get("pending_install_names", []))
    today_pending_names = {item["name"] for item in pending_installations}

    newly_completed_names = yesterday_pending_names - today_pending_names
    newly_completed_count = len(newly_completed_names)

    if snapshot_date == today_str:
        installations_done = int(yesterday.get("installations_done", 0)) + newly_completed_count
    else:
        installations_done = newly_completed_count

    installations_carried = len(yesterday_pending_names & today_pending_names)

    BLOCKER_PATTERNS = [
        ("client unreachable", ["no response", "not responding", "unreachable", "not picking",
                                "not answering"]),
        ("LAN / network",      ["lan", "network", "internet", "connectivity", "wifi", "wi-fi"]),
        ("machine not ready",  ["machine not installed", "machine not ready", "hardware", "not yet
                                installed"]),
        ("IT availability",    ["it manager", "it team", "it person", "tech person"]),
        ("awaiting details",   ["machine details", "ip address", "credentials", "details
                                pending"]),
        ("rescheduled",       ["scheduled for", "connect at", "call tomorrow", "follow up"]),
    ]

    enriched = []
    for item in pending_installations:
        msg_lower = (item.get("message") or "").lower()
        blocker = ""
        for label, keywords in BLOCKER_PATTERNS:
            if any(kw in msg_lower for kw in keywords):

```

```

        blocker = label
        break
    enriched.append({
        **item,
        "carried": item["name"] in yesterday_pending_names,
        "blocker": blocker,
        "status": "pending",
    })

return enriched, installations_done, installations_carried

```

— Tech conversation summariser —————

```

def summarize_tech_conversations(tech_rows: list, api_key: str) -> str:
    if not api_key or not tech_rows:
        return ""

    snippets = []
    for row in tech_rows:
        content = (row.get("content") or "").strip()
        sub      = (row.get("sub_category") or "").strip()
        if content:
            snippets.append(f"[{sub}] {content[:300]}")
        if len(snippets) >= 20:
            break

    if not snippets:
        return ""

    combined = "\n---\n".join(snippets)

    prompt = f"""
You are analyzing customer support ticket conversations from a teleradiology platform.

Below are up to 20 ticket conversation excerpts from today, grouped by sub-category.

---
{combined}
---

Identify the major technical issues from these teleradiology support conversations.
IGNORE metadata like 'call duration', 'call answered', 'call ended', 'incoming call'.
Focus on PACS issues, image sending failures, portal access, and radiologist connectivity.

Rules:
- Identify the 3 most important unique technical issues.
- For each, provide a short 5-10 word description of the problem.
- No counts, no names, no dates.
"""

    result = call_llm(prompt, api_key, max_tokens=250, expect_json=False)
    return result or ""

```

— LLM Core —————

```

def call_llm(prompt, api_key, max_tokens=1600, expect_json=True):
    if not api_key:
        return None
    try:
        response = requests.post(
            "https://api.openai.com/v1/chat/completions",
            headers={"Content-Type": "application/json", "Authorization": f"Bearer {api_key}"},
            json={
                "model": "gpt-4.1-mini",
                "messages": [{"role": "user", "content": prompt}],
                "temperature": 0.2,
            }
        )
    except:
        return None
    if not response:
        return None
    if not response.json():
        return None
    return response.json().get("choices", [{}])[0].get("message", {}).get("content", "")

```

```

        "max_tokens": max_tokens
    },
    timeout=50
)
response.raise_for_status()
content = response.json()["choices"][0]["message"]["content"].strip()
if not content:
    return None
if expect_json:
    clean = content
    if "```json" in clean:
        clean = clean.split("```json")[1].split("```")[0].strip()
    elif "```" in clean:
        clean = clean.split("```")[1].split("```")[0].strip()
    return json.loads(clean)
return content
except Exception as e:
    print(f" LLM Error: {e}")
    return None

```

— Master AI prompt —————

```

def generate_all_insights(data, api_key):
    esc_lines = []
    for spec in data["specialists"]:
        for e in spec.get("escalations", []):
            esc_lines.append(
                f" {spec['name']} | {e.get('issue','')} | {e.get('details','')[:100]}"
            )
    esc_text = "\n".join(esc_lines) if esc_lines else " None recorded"
    inst_text = " | ".join([
        f"{p['name']} (carried={p.get('carried',False)}, blocker={p.get('blocker','?')})"
        for p in data["pending_installations"]
    ]) if data["pending_installations"] else "None"

    d = data.get("deltas", {})
    delta_ctx = (
        f" Volume Δ: {d.get('total_vs_yesterday',0):+} (XR {d.get('xray_delta',0):+}, "
        f"CT {d.get('ct_delta',0):+}, MRI {d.get('mri_delta',0):+})\n"
        f" Supply Δ: {d.get('supply_delta',0):+}pp IQC Δ: {d.get('iqc_delta',0):+}pp OQC Δ: "
        f"{d.get('oqc_delta',0):+}pp\n"
        f" Tickets Δ: {d.get('tickets_delta',0):+} Tech Δ: {d.get('tech_delta',0):+} Sales Δ: "
        f"{d.get('sales_delta',0):+}"
    ) if d else " No comparison available."

```

prompt = f"""

You are a Senior Operations Analyst writing a daily briefing email for C-suite at 5C Network, a teleradiology platform.

STRICT RULES:

- SENTENCE LIMITS ARE HARD CAPS: 2-sentence fields = exactly 2. 1-sentence = exactly 1.
- Executive tone. COO/CEO audience. Confident, direct, zero filler words.
- Be realistic and critical: If metrics are flat or acquisition is zero, interpret it as a "plateau", "stagnation", or "risk" rather than being over-optimistic.
- NEVER restate raw numbers – interpret what they mean for the business.
- Where deltas exist, weave direction into insight ("down for second day", "recovering from yesterday's dip").
- Plain prose. No bullets, no markdown, no em-dashes as list separators.
- Risk flag detail: must name a specific business consequence, not just the metric.

TODAY ({data['report_date']})

VOLUME: Total {data['case_counts']['total']} | XRAY {data['case_counts']['xray']} | CT

```
{data['case_counts']['ct']} | MRI {data['case_counts']['mri']} | NM {data['case_counts']['nm']}  
SEGMENTS: {"", ".join([f"{g['name']} ({g['total']})" for g in data['client_groups']])}
```

```
DELTAS VS YESTERDAY:  
{delta_ctx}
```

```
SUPPLY {data['supply']['performance']}% | IQC {data['delivery']['iqc_percentage']}% | OQC  
{data['delivery']['oqc_percentage']}%
```

```
SALES: {data['onboarded_count']} new onboardings | {data['reactivation_count']} reactivated clients  
| {len(data['pending_platform_demo_clients'])} platform demos pending  
NEW CLIENTS: {"", ".join(data['onboarded_clients']) or "none"} | REACTIVATED: {"",  
".join(data['reactivation_clients']) or "none"} | DEMOS PENDING: {"",  
".join(data['pending_platform_demo_clients']) or "none"}
```

```
INSTALLATIONS: {data['installations_done']} done today | {len(data['pending_installations'])}  
pending | {data['installations_carried']} carried from yesterday  
PENDING: {inst_text}
```

```
TICKETS: Total {data['ticket_closure']['total']} | Details: {data.get('ticket_details_for_ai',  
'None')}  
FINANCE PENDING: {data['finance_pending']}  
CUSTOMER INTERACTIONS: AURA {data.get('aura_count', 0)} clients | Nexus {data.get('nexus_count',  
0)} clients | Tech Issues {data['tech_count']} tickets | Categories: {"",  
".join(data['raw_tech_categories'][:6]) or "none"}  
SUMMARIZED TECH CONVERSATIONS: {data.get('tech_conversation_insight', 'none')}  
DEMO CASES: {" | ".join(data['demo_sentences'][:5]) or "none"}
```

```
ESCALATIONS:  
{esc_text}
```

Return ONLY valid JSON (no markdown, no preamble):

```
{{  
  "headline": "One punchy sentence ≤10 words. No jargon.",  
  "executive_summary": "EXACTLY 2 sentences. S1: overall volume variations (summarize flat volume  
as a plateau). S2: How it can be improved.",  
  "volume_insight": "EXACTLY 2 sentences. S1: which and all segments have provided more case  
volume. S2: whether there is a growth or decline.",  
  "supply_delivery_insight": "EXACTLY 2 sentences. S1: Rate at which cases were pulled from supply  
queue (highlight lack of improvement if flat). S2: how supply pull performance plus IQC/OQC  
interplay impacts overall delivery efficiency.",  
  "reworks_qc_insight": "EXACTLY 1 sentence. Mention the main cause for reworks and QC rejects and  
the operational implication.",  
  "sales_insight": "EXACTLY 1 sentence. Interpret what the new onboardings, reactivations, and  
pending platform demos together signal (mention specific clients or note lack thereof).",  
  "customer_interactions_insight": "A concise paragraph (2-3 sentences). Summarize the main  
technical issues identified in the 'SUMMARIZED TECH CONVERSATIONS' section. STERNLY IGNORE metadata  
like 'call duration', 'call answered', 'call ended', or 'incoming call'. Describe only the actual  
technical problems radiologists or centers faced.",  
  "installation_insight": "EXACTLY 1 sentence. State how many are carried over and the impact of  
deployment delays.",  
  "ticket_insight": "EXACTLY 1 sentence. Summarize the sub-categories of closed tickets.",  
  "tech_insight": "EXACTLY 1 sentence. Stable or name the issue with delta direction.",  
  "demo_insight": "EXACTLY 1 sentence. Near-term conversion risk or opportunity.",  
  "escalation_insight": "EXACTLY 2 sentences. S1: common pattern. S2: compounding risk if  
unresolved.",  
  "risk_flags": [  
    {"label": "2-4 word noun phrase", "detail": "1 sentence: specific business consequence.",  
"level": "high"}},  
    {"label": "2-4 word noun phrase", "detail": "1 sentence: specific business consequence.",  
"level": "medium"}},  
    {"label": "2-4 word noun phrase", "detail": "1 sentence: specific business consequence.",  
"level": "low"}  
  ]  
}}
```

```

],
"closing_action": "EXACTLY 1 sentence. Imperative. Name a person/team/system. Address highest-
risk flag."
}}
"""
result = call_llm(prompt, api_key, max_tokens=1400, expect_json=True)
if result:
    return result

return {
    "headline": f"Ops briefing – {data['report_date']}",
    "executive_summary": "All systems operational with mixed SLA performance. Carry-over
installations and SLA gaps are the priority risks today.",
    "volume_insight": f"Platform processed {data['case_counts']['total']} studies with X-Ray
dominating throughput mix. Higher CT and MRI share would improve per-case margin.",
    "supply_delivery_insight": f"Supply pull rate at {data['supply']['performance']}% with IQC
and OQC performance diverging from target. Sustained gap between supply throughput and QC
completion will widen TAT exposure for high-volume clients.",
    "reworks_qc_insight": "Elevated reworks and QC rejects suggest inconsistency in case
routing to under-specialised readers, signalling a need for targeted radiologist feedback.",
    "sales_insight": "New client onboardings reflect active conversion while reactivations
signal recovering relationships dont use pipeline term." if data['onboarded_count'] > 0 or
data['reactivation_count'] > 0 else "Zero new onboardings and no reactivations today signals
decline in sales outreach.",
    "customer_interactions_insight": f"{data.get('aura_count', 0)} clients reached out via AURA
and {data.get('nexus_count', 0)} via Nexus today, indicating active client engagement that warrants
prompt resolution.",
    "installation_insight": f"{data['installations_carried']} installations carried from
yesterday with client unavailability as the dominant blocker, risking delayed revenue activation.",
    "ticket_insight": f"{data['ticket_closure']['total']} tickets closed today across multiple
categories.",
    "tech_insight": "Technical systems operating without critical incidents though unresolved
categories warrant monitoring.",
    "demo_insight": "Demo pipeline is active – conversion follow-up within 24 hours is needed
to avoid lead decay.",
    "escalation_insight": "Escalations share a pattern of unreachable contacts blocking
resolution loops. Unresolved today, these cases risk client trust degradation and potential
churn.",
    "risk_flags": [
        {"label": "Carry-over Installs", "detail": "Unresolved installations delay go-live
revenue and create client dissatisfaction.", "level": "high"},
        {"label": "IQC SLA Gap", "detail": "IQC below target risks client-facing TAT
breaches and contract penalties.", "level": "medium"},
        {"label": "Finance Pending", "detail": "Pending approvals may delay new client
activation and cash flow.", "level": "low"},
    ],
    "closing_action": "Installation team to contact all carry-over clients before noon and
confirm go-live timeline today."
}

```

— Specialist helpers _____

```

def score_escalation_message(msg):
    if not msg: return 0
    m = msg.lower()
    score = 0
    # User's specified major keywords
    if 'over called' in m: score += 12
    if 'significant' in m: score += 12
    if 'non-significant' in m or 'non significant' in m: score += 5
    if 'dislike' in m: score += 10

    # Priority indicators
    if '@' in m: score += 8
    if 'urgent' in m or 'critical' in m: score += 10

```

```

if 'tat' in m or 'delay' in m: score += 7
if 'error' in m or 'incorrect' in m or 'wrong' in m: score += 7
if 'complaint' in m or 'dissatisfied' in m or 'issue' in m: score += 6

# Keywords indicating real issues
major_kw = ['reject','over called', 'dispute','significant', 'tat','non-
significant','portal','dislike','pacs', 'missing',
'failed','issue','complaint','dissatisfied','delay','breached','critical']
for kw in major_kw:
    if kw in m:
        score += 5

# Filter out trivial/routine updates
trivial = [
    'retried', 'retry', 'unreachable', 'switched off', 'call later', 'follow up', 'checking',
    'appropriate point of contact', 'relevant contact details', 'not the appropriate',
    'share the relevant', 'contact details', 'point of contact'
]
for t in trivial:
    if t in m:
        score -= 20

return score

def extract_escalations_for_specialist(messages, specialist_name, api_key):
    if not messages or not api_key:
        return []
    combined = "\n---\n".join(messages)
    prompt = f"""
Analyze these communication logs for {specialist_name}. Extract only distinct major escalations or
significant operational issues.

Focus on: technical failures, client requests, TAT issues, workflow gaps,major issues or clear
complaints.
IGNORE purely routine updates, administrative notes, "wrong person contacted" messages, and
internal routing logs.

Keywords to look for: 'reject', 'over called', 'critical','delay','dissatisfied','significant',
'tat','breached', 'non-significant', 'portal', 'dislike', 'pacs', 'missing',
'failed','issue','complaint','dissatisfied','delay','breached','critical','fracture'.

Dont consider messages about:
- "Not the appropriate point of contact" / "Wrong contact person"
- "Will share relevant contact details"
- Retrying calls or internal routing updates
- Routine follow-ups without an operational blocker or failure

IMPORTANT: If the message contains @mentions (e.g. @Rahul, @team), include them in the 'details'
field as tags.
Return ONLY valid JSON.
If there are no actual escalations or significant issues (e.g. only routine updates), return an
empty list [] for 'top_escalations'.

DATA:
---
{combined[:3000]}
---

Return ONLY valid JSON:
{
    "specialist_summary": {
        "name": "{specialist_name}",
        "top_escalations": [
            {
                "issue": "Concise title",

```



```

        "details": "Summary including context and @tags.",
        "specialist_name": "{specialist_name}"
    }}
]
}}
}
"""
result = call_llm(prompt, api_key, max_tokens=1000, expect_json=True)
if result and isinstance(result, dict):
    return result.get("specialist_summary", {}).get("top_escalations", [])
return []

def summarize_installation_logs(client_name, logs, api_key):
    if not logs or not api_key:
        return "Installation pending – no recent updates."
    prompt = f"""
Summarize installation status for "{client_name}" in ONE professional sentence. State current
status and next action. Remove @mentions and informal language.
Logs: {chr(10).join(logs)[:500]}
Return ONLY the sentence – no labels, no preamble.
"""
    return call_llm(prompt, api_key, max_tokens=80, expect_json=False) or "Installation status
update pending."

# — AURA count helper —————
def get_aura_count(aura_conn) -> int:
    """
    Runs aura.sql which returns COUNT(DISTINCT client_id) AS client_id.
    Reads the numeric value from the first (and only) row, first column.
    Falls back to 0 on any error.
    """
    if not aura_conn:
        print(" x AURA connection unavailable – count = 0")
        return 0
    try:
        rows = pg_query(aura_conn, "aura.sql")
        if not rows:
            print(" x AURA query returned no rows – count = 0")
            return 0
        # The SQL aliases the count as 'client_id', value is an integer count
        first_row = rows[0]
        # Get the first value regardless of column name
        count_val = list(first_row.values())[0]
        count = int(count_val)
        print(f" ✓ AURA: {count} distinct clients")
        return count
    except Exception as e:
        print(f" x AURA count error: {e}")
        return 0

# — Main —————
def main():
    print("\n— Connecting —————")
    crm_conn = pg_connect(
        host=config.get("DB_HOST"), port=int(config.get("DB_PORT", 5432)),
        dbname=config.get("DB_NAME"), user=config.get("DB_USER"),
        password=config.get("DB_PASSWORD")
    )
    yaake_conn = pg_connect(
        host=config.get("YAAKE_HOST"), port=int(config.get("YAAKE_PORT", 5432)),
        dbname=config.get("YAAKE_DB"), user=config.get("YAAKE_USER"),
        password=config.get("YAAKE_PASSWORD")
    )

```

```

ch_client = ch_connect()
api_key   = config.get("API_KEY")

print("\n— Fetching data —————")

# Case counts
case_rows = ch_query(ch_client, "case_count.sql")
case_counts = {"total": 0, "xray": 0, "ct": 0, "mri": 0, "nm": 0}
for row in case_rows:
    m = str(row[0]).upper(); n = row[1]
    if m == "XRAY": case_counts["xray"] = n
    elif m == "CT": case_counts["ct"] = n
    elif m == "MRI": case_counts["mri"] = n
    elif m == "NM": case_counts["nm"] = n
case_counts["total"] = sum(v for k, v in case_counts.items() if k != "total")

# Client groups
cg_map = {}
for row in ch_query(ch_client, "client_group.sql"):
    grp = row[0]; mod = str(row[1]).upper(); cnt = row[2]
    if grp not in cg_map:
        cg_map[grp] = {"name": grp, "total": 0, "xray": 0, "ct": 0, "mri": 0, "nm": 0}
    if mod == "XRAY": cg_map[grp]["xray"] = cnt
    elif mod == "CT": cg_map[grp]["ct"] = cnt
    elif mod == "MRI": cg_map[grp]["mri"] = cnt
    elif mod == "NM": cg_map[grp]["nm"] = cnt
    cg_map[grp]["total"] += cnt
client_groups = sorted(list(cg_map.values()), key=lambda x: x['total'], reverse=True)

# SME data
sme_rows = ch_query(ch_client, "sme.sql")
sme_data = []
for row in sme_rows:
    sme_data.append({
        "client_name": row[0],
        "xray": row[1],
        "ct": row[2],
        "mri": row[3],
        "nm": row[4],
        "today": row[5],
        "yesterday": row[6],
        "last_week_this_day": row[7],
    })

# Supply Pull
supply = {"performance": 0.0, "cases_lt_10": 0, "total_cases": 0}
supply_pull = {"total": 0, "lt_10": 0, "lt_20": 0, "lt_30": 0}
rows = ch_query(ch_client, "supply.sql")
if rows:
    r = rows[0]
    supply_pull["lt_10"] = int(r[0] or 0)
    supply_pull["lt_20"] = int(r[1] or 0)
    supply_pull["lt_30"] = int(r[2] or 0)
    supply_pull["total"] = int(r[3] or 0)

supply_wait = {"total": 0, "lt_15": 0, "lt_30": 0, "lt_60": 0}
rows = ch_query(ch_client, "supply_detailed.sql")
if rows:
    r = rows[0]
    supply_wait["total"] = int(r[0] or 0)
    supply_wait["lt_15"] = int(r[1] or 0)
    supply_wait["lt_30"] = int(r[2] or 0)
    supply_wait["lt_60"] = int(r[3] or 0)

# Delivery Pull

```

```

delivery_pull = {
    "iqc_lt_10": 0, "iqc_lt_20": 0, "iqc_lt_30": 0, "iqc_total": 0,
    "oqc_lt_10": 0, "oqc_lt_20": 0, "oqc_lt_30": 0, "oqc_total": 0
}
rows = ch_query(ch_client, "delivery.sql")
if rows:
    r = rows[0]
    delivery_pull["iqc_total"] = int(r[0] or 0)
    delivery_pull["iqc_lt_10"] = int(r[1] or 0)
    delivery_pull["iqc_lt_20"] = int(r[2] or 0)
    delivery_pull["iqc_lt_30"] = int(r[3] or 0)
    delivery_pull["oqc_total"] = int(r[5] or 0)
    delivery_pull["oqc_lt_10"] = int(r[6] or 0)
    delivery_pull["oqc_lt_20"] = int(r[7] or 0)
    delivery_pull["oqc_lt_30"] = int(r[8] or 0)

delivery_wait = {
    "oqc_lt_15": 0, "oqc_lt_30": 0, "oqc_lt_60": 0, "oqc_total": 0,
    "iqc_lt_15": 0, "iqc_lt_30": 0, "iqc_lt_60": 0, "iqc_total": 0,
    "oqc_percentage": 0.0, "iqc_percentage": 0.0
}
rows = ch_query(ch_client, "delivery_detailed.sql")
if rows:
    r = rows[0]
    delivery_wait["oqc_total"] = int(r[0] or 0)
    delivery_wait["oqc_lt_15"] = int(r[1] or 0)
    delivery_wait["oqc_lt_30"] = int(r[2] or 0)
    delivery_wait["oqc_lt_60"] = int(r[3] or 0)
    delivery_wait["iqc_total"] = int(r[4] or 0)
    delivery_wait["iqc_lt_15"] = int(r[5] or 0)
    delivery_wait["iqc_lt_30"] = int(r[6] or 0)
    delivery_wait["iqc_lt_60"] = int(r[7] or 0)

# HIL Pull
hil_pull = {"total": 0, "lt_10": 0, "lt_20": 0, "lt_30": 0}
rows = ch_query(ch_client, "hil.sql")
if rows:
    r = rows[0]
    hil_pull["total"] = int(r[0] or 0)
    hil_pull["lt_10"] = int(r[1] or 0)
    hil_pull["lt_20"] = int(r[2] or 0)
    hil_pull["lt_30"] = int(r[3] or 0)

hil_wait = {"total": 0, "lt_15": 0, "lt_30": 0, "lt_60": 0}
rows = ch_query(ch_client, "hil_detailed.sql")
if rows:
    r = rows[0]
    hil_wait["total"] = int(r[0] or 0)
    hil_wait["lt_15"] = int(r[1] or 0)
    hil_wait["lt_30"] = int(r[2] or 0)
    hil_wait["lt_60"] = int(r[3] or 0)

delivery = delivery_wait
supply_pending = {
    "total": supply_wait["total"],
    "gt_30": supply_wait["total"] - supply_wait["lt_30"],
    "gt_60": supply_wait["total"] - supply_wait["lt_60"]
}
hil_pending = {
    "total": hil_wait["total"],
    "gt_30": hil_wait["total"] - hil_wait["lt_30"],
    "gt_60": hil_wait["total"] - hil_wait["lt_60"]
}

# Sales

```

```

onboarded_clients = []
for row in pg_query(yaake_conn, "onboarded.sql"):
    name = (row.get("onboarded_count") or row.get("client_name")
            or row.get("name") or (list(row.values())[0] if row else None))
    if name:
        onboarded_clients.append(str(name))
reactivation_clients = [
    r.get("client_name") for r in pg_query(yaake_conn, "reactivation.sql")
    if r.get("client_name")
]
pending_platform_demo_clients = [
    r.get("client_name") for r in pg_query(yaake_conn, "pending_platform_demo.sql")
    if r.get("client_name")
]

# Rad Onboardings
rad_onboarded_count = 0

rad_lead_rows = pg_query(yaake_conn, "rad_lead.sql")
rad_lead_count = int(list(rad_lead_rows[0].values())[0]) if rad_lead_rows else 0

# QC Details
qc_leave_rows = ch_query(ch_client, "qc_leave.sql")
qc_leave_count = int(qc_leave_rows[0][0]) if qc_leave_rows else 0

qc_ot_rows = pg_query(yaake_conn, "qc_overtime.sql")
qc_ot_count = int(list(qc_ot_rows[0].values())[0]) if qc_ot_rows else 0

# Tickets
total_closed_rows = pg_query(crm_conn, "total_closed.sql")
sub_grouped = {}
for r in total_closed_rows:
    cat = r.get("category", "")
    sub = r.get("sub_category", "")
    cnt = int(r.get("closed_ticket_count", 0))
    # Keep only sub-category for display as per request
    label = sub if sub else cat
    sub_grouped[label] = sub_grouped.get(label, 0) + cnt

total_closed_categories = [
    {"category": label, "closed_ticket_count": cnt}
    for label, cnt in sorted(sub_grouped.items(), key=lambda x: -x[1])
]
ticket_details_for_ai = ", ".join(
    [f"{k} ({v})" for k, v in sorted(sub_grouped.items(), key=lambda x: -x[1])]
)
ticket_closure = {
    "total": sum(sub_grouped.values()),
    "client": sum(sub_grouped.values()),
    "rad": 0,
}

# Finance
fin_rows = pg_query(yaake_conn, "fianance_approval.sql")
finance_pending = fin_rows[0].get("count", 0) if fin_rows else 0

# Tech issues
tech_rows = pg_query(crm_conn, "tech_issue.sql")
tech_count = sum(r.get("ticket_count", 0) for r in tech_rows)
raw_tech_categories = [str(r["sub_category"]) for r in tech_rows if r.get("sub_category")]
tech_sub_breakdown = [
    {"sub_category": r.get("sub_category", ""), "count": int(r.get("ticket_count", 0))}
    for r in sorted(tech_rows, key=lambda x: -int(x.get("ticket_count", 0)))
]

```

```

        if r.get("sub_category")
    ]

    print(" Summarising tech conversations...")
    tech_conversation_insight = summarize_tech_conversations(tech_rows, api_key)
    print(f" ✓ Tech insight: {tech_conversation_insight[:80]}..." if tech_conversation_insight
else " ✓ Tech insight: none")

# — AURA count —————
aura_count = 0

# — Nexus count —————
nexus_rows = pg_query(crm_conn, "nexus.sql")
nexus_count = len(nexus_rows)
print(f" ✓ Nexus: {nexus_count} distinct clients")

# Customer interactions summary dict (used by template and AI prompt)
customer_interactions = {
    "aura_count":    aura_count,
    "nexus_count":   nexus_count,
    "total":         aura_count + nexus_count,
    "tech_breakdown": tech_sub_breakdown,
    "tech_total":    tech_count,
}

# Reworks and QC Rejects
reworks_rows = ch_query(ch_client, "reworks.sql")
reworks_total = 0
top_reworks_clients = []
top5_reworks_clients = []
if reworks_rows:
    client_rework_map = {}
    for row in reworks_rows:
        client_name = row[2] or f"Client {row[0]}"
        count = int(row[1] or 0)
        client_rework_map[client_name] = client_rework_map.get(client_name, 0) + count
    sorted_reworks = sorted(client_rework_map.items(), key=lambda x: -x[1])
    reworks_total = sum(v for _, v in client_rework_map.items())
    top_reworks_clients = [{"name": n, "count": c} for n, c in sorted_reworks[:3]]
    top5_reworks_clients = [{"name": n, "count": c} for n, c in sorted_reworks[:5]]
reworks_count = reworks_total
qc_reject_rows = ch_query(ch_client, "qc_reject.sql")
qc_reject_count = int(qc_reject_rows[0][0]) if qc_reject_rows else 0

# — X-Ray Only & No-Case Clients —————
sme_names = {row["client_name"] for row in sme_data}

# Process X-Ray Only
raw_xray_rows = ch_query(ch_client, "xray_only.sql")
filtered_xray = []
for row in raw_xray_rows:
    name = str(row[0] or "")
    if name not in sme_names:
        filtered_xray.append({
            "client_name": name,
            "xray_today": int(row[5] or 0),
            "last_week": int(row[6] or 0)
        })
filtered_xray.sort(key=lambda x: x["last_week"], reverse=True)
xray_only_clients = filtered_xray[:5]

# Process No-Case Clients
raw_no_case_rows = ch_query(ch_client, "no_case_clients.sql")
filtered_no_case = []
for row in raw_no_case_rows:

```

```

        name = str(row[0] or "")
        if name not in sme_names:
            filtered_no_case.append({
                "client_name": name,
                "today":      int(row[5] or 0),
                "last_week":  int(row[6] or 0)
            })
    filtered_no_case.sort(key=lambda x: x["last_week"], reverse=True)
    no_case_clients = filtered_no_case[:5]

# Pending installations
grouped_logs = {}
for row in pg_query(yaake_conn, "pending_installation.sql"):
    name = row.get("lead_name") or row.get("client_name") or row.get("name")
    msg  = row.get("original_message") or ""
    if name:
        grouped_logs.setdefault(name, []).append(msg)
print(" Summarising installation logs...")
raw_pending = [
    {"name": name, "message": summarize_installation_logs(name, logs, api_key)}
    for name, logs in grouped_logs.items()
]

# Specialists
specialist_messages = {}
for row in pg_query(yaake_conn, "solution.sql"):
    spec = str(row.get("user_name") or row.get("note_author") or "").strip()
    msg   = str(row.get("original_message") or "").strip()
    updater = str(row.get("user_name") or "System").strip()
    if spec:
        payload = f"Author: {updater} | Message: {msg}"
        specialist_messages.setdefault(spec, []).append(payload)

specialist_demo = {}
for row in ch_query(ch_client, "solution_demo.sql"):
    spec = str(row[1] or "").strip()
    if spec:
        specialist_demo.setdefault(spec, []).append({
            "case_id": row[0], "client_fk": row[2], "tat_min": row[3],
            "client_name": row[4], "modality": row[5], "demo_count": row[6]
        })

incubation_map = {
    str(row[1] or "").strip(): int(row[0] or 0)
    for row in ch_query(ch_client, "incubation.sql") if row[1]
}

activity_map = {}
for row in ch_query(ch_client, "client_details.sql"):
    spec = str(row[0] or "").strip()
    client_name = str(row[1] or "").strip()
    status = str(row[2] or "").strip()
    if spec not in activity_map:
        activity_map[spec] = {"active": 0, "inactive": 0, "total": 0, "inactive_clients": []}
    activity_map[spec]["total"] += 1
    if status == "Active":
        activity_map[spec]["active"] += 1
    else:
        activity_map[spec]["inactive"] += 1
        if client_name:
            activity_map[spec]["inactive_clients"].append(client_name)

SPECIALISTS = sorted(list(specialist_messages.keys()))
print(" Extracting escalations...")

```

```

solution_specialists = []
for spec in SPECIALISTS:
    messages = specialist_messages.get(spec, [])
    demos = specialist_demo.get(spec, [])

    scored_messages = sorted(
        [(score_escalation_message(m), m) for m in messages],
        key=lambda x: x[0], reverse=True
    )
    top_messages = [m[1] for m in scored_messages[:15]]
    max_score = scored_messages[0][0] if scored_messages else 0

    extracted = extract_escalations_for_specialist(top_messages, spec, api_key) if top_messages
else []
    for e in extracted:
        e["priority_score"] = max_score

    escalations = extracted

    activity = None
    for k, v in activity_map.items():
        if k in spec or spec in k:
            activity = v
            break

    demo_clients = {}
    demo_cases = []
    for dm in demos:
        cn = dm["client_name"] or f"Client {dm['client_fk']}"
        demo_clients[cn] = demo_clients.get(cn, 0) + dm["demo_count"]
        demo_cases.append({"modality": dm["modality"], "client_name": cn, "tat_min":
dm["tat_min"]})

    solution_specialists.append({
        "name": spec,
        "escalations": escalations,
        "demo_clients": demo_clients,
        "demo_cases": demo_cases,
        "total_demos": sum(demo_clients.values()),
        "incubation_count": incubation_map.get(spec, 0),
        "activity": activity,
    })

grouped_demos = {}
for row in ch_query(ch_client, "demo_clients.sql"):
    if len(row) < 4: continue
    grouped_demos.setdefault((row[1], row[2]), []).append(str(int(row[3] or 0)))
demo_sentences = [
    f"{len(tats)} {mod} case{'s' if len(tats)>1 else ''} · {c_name} · {'', '.join(tats)} min"
    for (c_name, mod), tats in grouped_demos.items()
]

print("\n— Computing deltas & enriching installations ————")
yesterday = load_snapshot()

full_data_pre = {
    "case_counts": case_counts,
    "supply": supply,
    "delivery": delivery,
    "ticket_closure": ticket_closure,
    "tech_count": tech_count,
    "onboarded_count": len(onboarded_clients),
    "reactivation_count": len(reactivation_clients),
    "pending_installations": raw_pending,
    "installations_done": 0,

```

```

}
deltas = compute_deltas(full_data_pre, yesterday)

print(" Fetching yesterday counts from SQL...")
yest_counts = {"total": 0, "xray": 0, "ct": 0, "mri": 0, "nm": 0}

deltas["total_vs_yesterday"] = case_counts["total"] - yest_counts["total"]
deltas["xray_delta"]          = case_counts["xray"]   - yest_counts["xray"]
deltas["ct_delta"]            = case_counts["ct"]     - yest_counts["ct"]
deltas["mri_delta"]           = case_counts["mri"]    - yest_counts["mri"]
deltas["nm_delta"]            = case_counts["nm"]     - yest_counts["nm"]

pending_installations, installations_done, installations_carried = enrich_installations(
    raw_pending, yesterday
)

print("\n— Generating AI insights (single call) —————")
all_escalations_pool = []
for spec in solution_specialists:
    for e in spec.get("escalations", []):
        e["specialist_name"] = spec["name"]
        all_escalations_pool.append(e)

all_escalations = []
by_spec = {}
for e in all_escalations_pool:
    by_spec.setdefault(e["specialist_name"], []).append(e)

leftovers = []
for sname, esc_list in by_spec.items():
    all_escalations.append(esc_list[0])
    leftovers.extend(esc_list[1:])

leftovers.sort(key=lambda x: x.get("priority_score", 0), reverse=True)

for e in leftovers:
    if len(all_escalations) >= 10:
        break
    all_escalations.append(e)

all_escalations = all_escalations[:10]

full_data = {
    "report_date":          datetime.now().strftime("%B %d, %Y"),
    "case_counts":          case_counts,
    "client_groups":        client_groups,
    "supply":               supply,
    "delivery":              delivery,
    "onboarded_clients":    onboarded_clients,
    "onboarded_count":      len(onboarded_clients),
    "reactivation_clients": reactivation_clients,
    "reactivation_count":   len(reactivation_clients),
    "pending_platform_demo_clients": pending_platform_demo_clients,
    "ticket_closure":       ticket_closure,
    "finance_pending":       finance_pending,
    "tech_count":            tech_count,
    "raw_tech_categories":   raw_tech_categories,
    "pending_installations": pending_installations,
    "installations_done":    installations_done,
    "installations_carried": installations_carried,
    "demo_sentences":        demo_sentences,
    "specialists":           solution_specialists,
    "deltas":                deltas,
    "ticket_details_for_ai": ticket_details_for_ai,
    "aura_count":            aura_count,

```



```

        "nexus_count":            nexus_count,
        "customer_interactions":  customer_interactions,
        "tech_conversation_insight": tech_conversation_insight,
    }
    ai = generate_all_insights(full_data, api_key)
    print("✓ AI insights ready")

    save_snapshot({
        **full_data_pre,
        "pending_installations": pending_installations,
        "installations_done":    installations_done,
    })

    print("\n— Rendering HTML —")
    print("  ✓ final.html ready")

    print("\n— Complete —\n")

if __name__ == "__main__":
    main()

```